VERIQTA

Cloud Security from Zero

A Beginner's Guide to Securing Your Cloud Infrastructure



@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

01. What Is Cloud Security?

VERIQTA

1. What Is Cloud Security?

- Cloud security means protecting your data, applications, infrastructure and users in the cloud.
- It involves tools, configurations, policies and best practices to prevent unauthorized access, data loss and service disruption.
- Engineers use cloud security to keep cloud resources safe, available and running as expected.

2. Why Does It Matter?

- You store sensitive data in the cloud.
- Your applications are accessible over the internet.
- Multiple people and services need access.
- Misconfigurations or stolen credentials can expose your resources.
- A single security mistake can lead to data breaches, financial loss and service outages.

3. What Do Engineers Protect in the Cloud?

Data	Compute	Networks	Applications	Users and Access
Customer data, business data, backups	Virtual machines, containers, serverless functions	VPCs, subnets, security groups, firewalls	Web apps, APIs, microservices	IAM users, roles, permissions, credentials

4. The Three Core Security Goals

Confidentiality

- Keep data private.
- Only authorized people can access it.
- Example: Use encryption and access controls to prevent unauthorized access.

Integrity

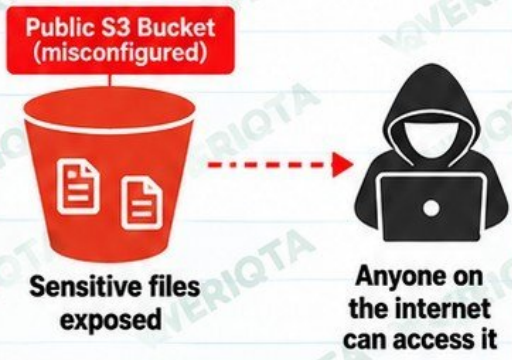
- Keep data accurate and unchanged.
- Prevent unauthorized modifications.
- Example: Use checksums, IAM policies and versioning to detect and prevent unintended changes.

Availability

- Keep services and data accessible when needed.
- Prevent downtime and disruptions.
- Example: Use redundancy, backups and multi-region deployments.

5. Real Example: An Exposed Cloud Resource

- A company accidentally made an Amazon S3 bucket public.
- Sensitive files, including customer data, were accessible by anyone on the internet.
- Attackers found the bucket and downloaded the data.
- This led to a data breach, financial loss and reputational damage.
- The issue was caused by a **misconfiguration,** not a complex hack.



Key Takeaway

- Cloud security is everyone's responsibility.
- Small mistakes can expose sensitive data, so always review access settings, use least privilege and follow security best practices.

02. The Shared Responsibility Model

VERIQTA 02

1. What Is the Shared Responsibility Model?

- The shared responsibility model explains which security tasks are handled by the cloud provider and which are your responsibility as a customer.
- The cloud provider secures the cloud infrastructure.
- You secure what you build, configure, and run in the cloud.
- Both sides must do their part for a secure environment.

2. What the Cloud Provider Protects

- Physical data centers, buildings, and facilities.
- Network infrastructure and hardware.
- Compute, storage, and networking hardware.
- Virtualization and the underlying cloud platform.
- Availability of the core cloud services.

aws

AWS secures the infrastructure that runs its cloud services.

3. What the Customer Must Protect

- Your data (e.g. databases, files, backups).
- Your applications and code.
- Identity and access management (users, roles, permissions).
- Security configurations (e.g. security groups, encryption).
- Operating systems, patches, and software (for IaaS).
- Network settings and firewalls (virtual networks, subnets).
- Monitoring, logging, and threat detection.



You are responsible for securing what you create and configure in the cloud.

4. How Responsibilities Differ Across IaaS, PaaS, and SaaS

Responsibility Area	IaaS (e.g. AWS EC2)	PaaS (e.g. AWS Elastic Beanstalk)	SaaS (e.g. AWS S3, Office 365)
Physical infrastructure	AWS	AWS	AWS
Network infrastructure	AWS	AWS	AWS
Compute, storage, virtualization	AWS	AWS	AWS
Operating system	You	AWS	AWS
Application runtime	You	AWS	AWS
Applications and code	You	You	You
Data and encryption	You	You	You
Identity and access management	You	You	You
Network security configuration	You	You	You
Monitoring and logging	You	You	You

5. AWS Shared Responsibility Example

aws

AWS is responsible for:

- Running and securing AWS data centers.
- Maintaining the underlying hardware.
- Keeping the core AWS services available.
- Patching and securing the cloud platform.

You are responsible for:

- Securing your EC2 instances (OS, patches).
- Configuring security groups and VPCs.
- Managing IAM users, roles and permissions.
- Protecting your data (S3 buckets, databases).
- Enabling logging and monitoring (CloudTrail, CloudWatch).
- Securing your applications and secrets.

Key Takeaway

- The cloud provider secures the cloud.
- You secure what you run in the cloud.
- Understanding this model helps you avoid security gaps.

03. Understanding Cloud Security Risks

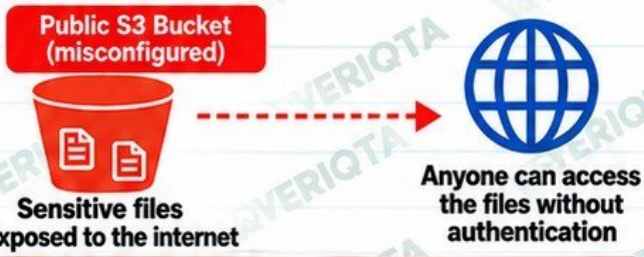
VERIQTA 03

1. What Are Cloud Security Risks?

- Cloud security risks are threats that can lead to unauthorized access, data loss, service disruption or financial loss in the cloud.
- These risks exist because cloud resources are accessible over the internet and are often misconfigured or not properly protected.
- Engineers must understand these risks to prevent security incidents.

2. Misconfigurations and Exposed Resources

- Misconfigurations happen when cloud resources are set up incorrectly.
- They can expose sensitive data or give unauthorized access.



Common Examples

- Public S3 buckets
- Overly open security group rules (e.g. 0.0.0.0/0)
- Exposed databases and management dashboards
- Default configurations left unchanged

3. Stolen Credentials and Excessive Permissions

- Attackers can steal credentials through phishing, malware or leaked keys.
- Excessive permissions make the damage worse because attackers can access more resources.

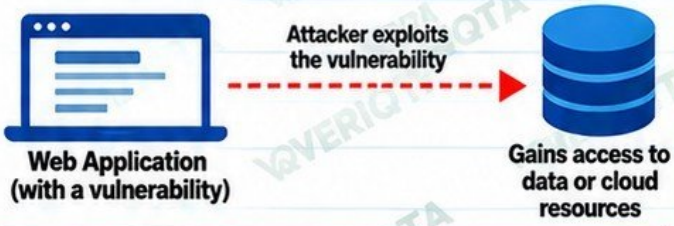


Common Problems

- Long-lived access keys
- Sharing credentials
- Using administrator permissions
- No multi-factor authentication (MFA)

4. Vulnerable Applications and Insecure APIs

- Applications and APIs in the cloud can have security weaknesses.
- Attackers can exploit these weaknesses to access data, steal credentials or take control of resources.



Common Examples

- Injection vulnerabilities (e.g. SQL injection)
- Unprotected or unauthenticated APIs
- Exposed admin panels
- Outdated software and unpatched components

5. How Small Mistakes Become Security Incidents

1 Small Mistake
- A resource is misconfigured.
- A credential is leaked.
- A security control is not enabled.

→

2 Increased Exposure
- Resource becomes publicly accessible.
- Attacker gains access with stolen credentials.
- Vulnerability is exploited.

→

3 Security Incident
- Data is accessed or stolen.
- Resources are modified or deleted.
- Services become unavailable.

→

4 Impact
- Financial loss.
- Reputational damage.
- Service outages.
- Loss of customer trust.

Key Takeaway

- Most cloud security incidents start with small, preventable mistakes.
- Always follow security best practices, review configurations and monitor your cloud environment.

04. Identity and Access Management (IAM)

VERIQTA 04

1. What Is IAM?

- Identity and Access Management (IAM) controls who can access your cloud resources and what they can do.
- It helps you manage users, roles, permissions and security policies.
- IAM is used to enforce least privilege, protect sensitive data and prevent unauthorized access.

2. Key IAM Components

Users
A user is a person or application that signs in to the cloud.

Roles
A role provides temporary permissions without long-term credentials.

Policies
A policy defines what actions are allowed or denied.

Permissions
Permissions are the specific actions you can perform on resources.

3. Authentication vs Authorization

Authentication
- Confirms who you are.
- Example: Username + password, MFA, or a security key.
- You cannot access the cloud until you are authenticated.

Authorization
- Determines what you are allowed to do.
- Controlled by IAM policies and permissions.
- Example: A user can read S3 objects but cannot delete them.

4. How Cloud Access Decisions Work



5. Example: IAM Policy

```
{
   "Version": "2012-10-17",
   "Statement": [
     {
          "Effect": "Allow",
          "Action": "s3:ListBucket",
          "Resource": "arn:aws:s3:::my-bucket"
     }
   ]
}
```

- This policy allows the user to list the contents of the S3 bucket named **my-bucket**.
- You can also deny actions using **"Effect": "Deny"**.
- Policies can be attached to users, groups or roles.

Least Privilege
- Only grant the permissions needed to perform a task.
- Avoid using administrator permissions unless necessary.
- Review and remove unused permissions regularly.

6. Why Engineers Use Temporary Credentials

- Temporary credentials are short-lived access keys provided through roles (e.g. IAM roles).
- They reduce the risk of stolen long-term credentials.
- Commonly used in EC2 instances, CI/CD pipelines and automation tools.
- They automatically expire, so the access is limited in time.



7. Common IAM Mistakes

- Using root account for daily work.
- Giving excessive permissions (e.g. AdministratorAccess).
- Not enabling MFA.
- Not rotating or removing unused access keys.
- Sharing credentials between people.
- Using long-term credentials in applications.

8. Key Takeaways

- IAM controls who can access cloud resources and what they can do.
- Always use least privilege.
- Use MFA for user accounts.
- Use roles and temporary credentials for applications and automation.
- Understand the difference between authentication and authorization.
- Regularly review and remove unused permissions.

05. Least Privilege and MFA

VERIQTA 05

1. What Is Least Privilege?

- Least privilege means giving users, roles or applications only the permissions they need to perform their tasks.
- It reduces the risk of accidental or malicious changes.
- If an account is compromised, the damage is limited.
- It is a core security principle in the cloud.

Example: Least Privilege in AWS

Administrator (too many permissions)

- Can create, delete and modify everything in the account.

Developer (least privilege)

- Can only access what they need (e.g. read S3, deploy to a specific environment).

2. What Is Multi-Factor Authentication (MFA)?

- MFA adds an extra layer of security during sign-in.
- You provide two or more forms of identification:
  - Something you know (password).
  - Something you have (authenticator app).
  - Something you are (biometrics like fingerprint).
- Even if a password is stolen, attackers cannot access your account without the second factor.

How MFA Works



3. Why Administrator Access Is Dangerous

- Administrator accounts have full access to all resources.
- If the credentials are stolen, attackers can create, delete or modify anything.
- It can lead to data breaches, service outages and financial loss.
- Administrators are often targeted by attackers.

⚠ Risks of Administrator Access

- Complete access to all resources.
- Attackers can create new accounts.
- Attackers can disable security controls.
- A single mistake can delete critical data.
- Harder to detect malicious activity.

4. Reviewing Permissions and Removing Unused Access

- Regularly review users, roles and permissions.
- Remove access that is no longer needed.
- Disable or delete unused accounts.
- Use cloud tools to find inactive or overly permissive access.
- This reduces your attack surface.

Example Review Process



5. Best Practices

- ✓ Always use least privilege.
- ✓ Enable MFA for all user accounts.
- ✓ Avoid using root or full administrator accounts for daily work.
- ✓ Use roles with limited permissions.
- ✓ Review and remove unused access regularly.
- ✓ Use automated tools to detect excessive permissions.

6. Key Takeaway

- Give only the permissions needed.
- Always use MFA.
- Avoid unnecessary administrator access.
- Regularly review and remove unused access.
- This keeps your cloud environment safe and reduces the impact of mistakes.

06. Securing Cloud Networks

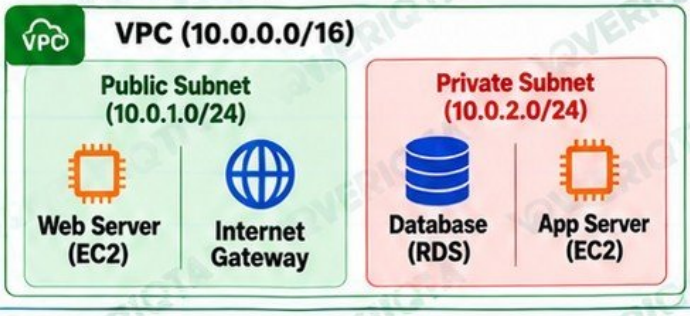
VERIQTA 06

1. What Is a Virtual Private Cloud (VPC)?

- A VPC is a logically isolated network in the cloud.
- It allows you to launch resources in a private network that you control.
- You can define IP address ranges, subnets, route tables and network security settings.
- It helps you keep your resources secure and separate from other customers.

2. Public and Private Submets

- A VPC is divided into subnets (smaller networks).
- Public subnets have a route to the internet (through an Internet Gateway).
- Private subnets do not have direct internet access.
- You typically place web servers in public subnets and backend services (databases, internal apps) in private subnets.



3. Security Groups

- Security groups act as a firewall for your instances.
- They control inbound and outbound traffic based on rules.
- You allow only the necessary traffic.
- Security groups are stateful (if you allow a request, the response is automatically allowed).

Example Security Group Rules (for a web server)

Type	Protocol	Port	Source
HTTP	TCP	80	0.0.0.0/0
HTTPS	TCP	443	0.0.0.0/0
SSH	TCP	22	Your IP only

4. Network Access Control Lists (NACLs)

- NACLs are an additional layer of security at the subnet level.
- They control inbound and outbound traffic for entire subnets.
- They are stateless (you must allow both the request and response).
- NACLs are useful for blocking traffic at the subnet boundary.

Example NACL Rules (for a public subnet)

Rule #	Type	Protocol	Port	Source/Destination	Action
100	Inbound	TCP	80	0.0.0.0/0	Allow
110	Inbound	TCP	443	0.0.0.0/0	Allow
120	Inbound	ALL	ALL	0.0.0.0/0	Deny
100	Outbound	ALL	ALL	0.0.0.0/0	Allow

5. Understanding Inbound and Outbound Traffic



Examples of inbound traffic:

- Web traffic (HTTP/HTTPS)
- SSH access
- API requests
- Traffic from a load balancer



Examples of outbound traffic:

- Software updates
- API calls to external services
- Sending logs and monitoring data
- Accessing package repositories

6. Best Practices

- Keep resources in private subnets when possible.
- Allow only required traffic in security groups and NACLs.
- Use least privilege for network access.
- Separate environments (dev, test, prod) using different subnets.
- Use VPC Flow Logs to monitor network traffic.
- Regularly review and remove unnecessary rules.

7. Common Mistakes

- Putting databases in public subnets.
- Allowing 0.0.0.0/0 for SSH (port 22).
- Overly permissive security group rules.
- Not using private subnets for internal services.
- Forgetting outbound traffic controls.
- Not monitoring network traffic.

07. Protecting Cloud Storage

VERIQTA 07

1. Public vs Private Object Storage

- Object storage can be public (accessible from the internet) or private (restricted access).
- Public buckets allow anyone to read the data without authentication.
- Private buckets require proper permissions to access the data.
- Always keep sensitive data (e.g. customer information, backups, logs) in private storage.

Public Bucket (anyone can access)



Anyone on the internet can read the files

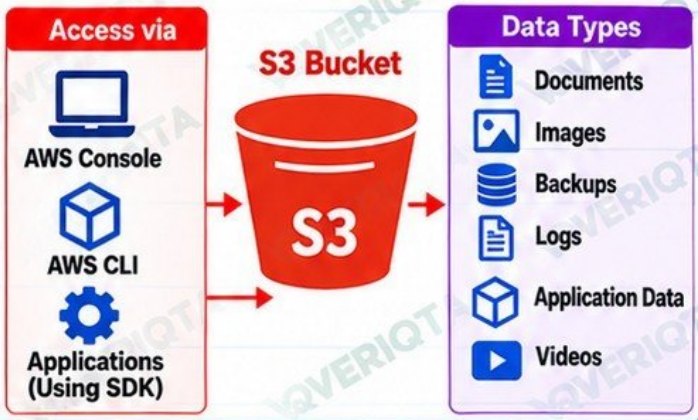
Private Bucket (restricted access)



Only authorized users can access the files

2. Amazon S3 Security Basics

- Amazon S3 is a scalable object storage service.
- By default, new buckets are private.
- You control access using bucket policies, IAM policies and access control settings.
- S3 can store any type of file (documents, images, backups, logs, application data).



3. Encryption and Access Policies

- Encrypt data at rest to protect it if unauthorized access occurs.
- Use HTTPS (encryption in transit) when transferring data.
- Control access with bucket policies and IAM policies.
- You can use AWS KMS to manage encryption keys.

Encryption at Rest



Data is encrypted when stored in S3 (using Amazon S3 encryption or AWS KMS)

Encryption in Transit



Data is encrypted using HTTPS/TLS when being uploaded or downloaded

4. Example: S3 Bucket Policy (Restrict Access)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": { "AWS": "arn:aws:iam::123456789012:role/AppRole" },
      "Action": [ "s3:GetObject", "s3:PutObject" ],
      "Resource": "arn:aws:s3:::my-bucket/*"
    }
  ]
}
```

What This Policy Does

- Allows only the specified IAM role to access the bucket.
- Grants permission to upload and download objects.
- Other users are denied access.

5. Preventing Accidental Data Exposure

- Always keep buckets private unless there is a valid reason to make them public.
- Disable public access settings at the bucket and account level.
- Review bucket policies regularly.
- Do not store sensitive data (passwords, keys, personal data) in public buckets.
- Use lifecycle policies to manage and delete old data.
- Enable logging and monitoring to detect unusual access.

6. Common Mistakes

- Leaving buckets publicly accessible.
- Using overly permissive bucket policies (e.g. "").
- Storing sensitive data without encryption.
- Not enabling versioning (accidental deletion).
- Not reviewing access permissions regularly.
- Sharing bucket URLs without access controls.

Key Takeaway

- Keep your S3 buckets private.
- Use encryption and proper access policies.
- Regularly review permissions and monitor access.

08. Encryption and Key Management

VERIQTA 08

1. What Is Encryption?

- Encryption converts readable data into an unreadable format (ciphertext).
- It protects your data from being read by unauthorized people.
- In the cloud, you should encrypt data both at rest and in transit.
- Encryption is a core security practice for all cloud workloads.

2. Encryption at Rest vs In Transit

Encryption at Rest



- Protects data stored on disk (e.g. S3, EBS, RDS).
- Data is encrypted even when not in use.
- Uses encryption keys (e.g. AWS KMS).

Encryption in Transit



- Protects data while it is being transmitted.
- Uses TLS/HTTPS to encrypt the connection.
- Prevents attackers from intercepting the data.

3. TLS and HTTPS

- TLS (Transport Layer Security) encrypts data between your client and the server.
- HTTPS is HTTP over TLS and is used for secure web communication.
- It protects sensitive information like passwords, tokens and API data from being intercepted.

HTTPS Connection



4. Encryption Keys and AWS KMS

- AWS Key Management Service (KMS) helps you create, manage and control encryption keys.
- You can use KMS keys to encrypt data in services like S3, EBS, RDS and others.
- KMS allows you to control who can use the keys through IAM policies.

How AWS KMS Works



Types of KMS Keys

AWS Managed Key

- Created and managed by AWS.
- Used for many AWS services.
- No additional cost.

Customer Managed Key

- Created and controlled by you.
- You define permissions.
- Used when you need more control.

5. Why Protecting Keys Matters

- If an encryption key is exposed, attackers can decrypt your data.
- Keys control access to your most sensitive information.
- Proper key management reduces the risk of data breaches.
- You should restrict who can use and manage keys, rotate keys regularly and monitor key usage.

Risks of Poor Key Management

- Unauthorized access to sensitive data.
- Attackers can decrypt stolen data.
- Loss of keys can make your data unavailable.
- Misconfigured permissions can expose keys.

6. Best Practices

- Always encrypt data at rest and in transit.
- Use HTTPS and TLS for all web services and APIs.
- Use AWS KMS for key management.
- Follow the principle of least privilege for key access.
- Rotate keys regularly.
- Monitor key usage with AWS CloudTrail and KMS logs.

7. Key Takeaway

- Encryption protects your data.
- Use TLS/HTTPS for data in transit.
- Use AWS KMS to manage encryption keys.
- Protecting keys is critical to keeping your data safe.

09. Secrets Management

VERIQTA 09

1. What Are Secrets?

- Secrets are sensitive pieces of information such as passwords, API keys, tokens, database credentials and certificates.
- They are used to authenticate and authorize access to cloud resources and services.
- Secrets must be protected from unauthorized access.
- Proper secrets management reduces the risk of data breaches.

2. Examples of Secrets

Passwords (e.g. database passwords)

API Keys (e.g. AWS access keys)

Tokens (e.g. OAuth tokens, session tokens)

Certificates (e.g. TLS certificates)

3. Why Secrets Must Not Be Stored in Git

- Git repositories are usually public or accessible to multiple people.
- Secrets in Git can be easily copied by attackers.
- Even if you delete a secret, it may still exist in previous commits.
- Exposed secrets can lead to unauthorized access, data theft and high cloud costs.

Example: Exposed Secret in Git

```
1  # Do not store secrets in Git
2  export DB_PASSWORD="MySecret123"
3  export API_KEY="AKIAIOSFODNN7EXAMPLE"
```

This should never be in Git

4. AWS Secrets Manager vs Parameter Store

AWS Secrets Manager

- Stores and manages sensitive data such as passwords, API keys and tokens.
- Automatic key rotation for supported services (e.g. RDS, Redshift).
- Higher cost but more features (e.g. rotation, auditing).
- Suitable for most production use cases.

AWS Systems Manager Parameter Store

- Stores configuration data and secrets.
- Supports encrypted parameters using AWS KMS.
- Lower cost than Secrets Manager.
- Suitable for simple secrets and configuration values.

5. How Applications Access Secrets Securely



- Applications use an IAM role (not hardcoded credentials) to access secrets.
- The request is made over a secure connection (HTTPS).
- The secret is retrieved at runtime and used in memory (not stored in code or logs).
- Access is controlled using IAM policies and least privilege.

6. Best Practices

- Never store secrets in Git or code.
- Use AWS Secrets Manager or Parameter Store.
- Encrypt secrets using AWS KMS.
- Use IAM roles with least privilege.
- Enable automatic rotation where possible.
- Do not log or print secrets in application logs.
- Monitor secret access using AWS CloudTrail.

7. Common Mistakes

- Storing secrets in Git repositories.
- Using hardcoded secrets in application code.
- Giving excessive IAM permissions to access secrets.
- Not rotating secrets regularly.
- Logging or exposing secrets in application logs.
- Using the same secret across multiple environments.

8. Key Takeaway

- Secrets are sensitive and must be protected.
- Never store secrets in Git.
- Use AWS Secrets Manager or Parameter Store.
- Grant only necessary permissions.
- Access secrets securely at runtime.
- Rotate secrets regularly and monitor access.

10. Securing Cloud Servers

VERIQTA 10

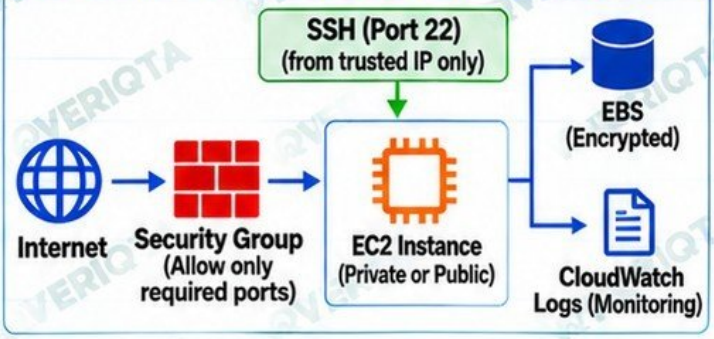
1. What Are Cloud Servers?

- Cloud servers (e.g. EC2 instances) are virtual machines in the cloud.
- You can choose the operating system, CPU, memory and storage.
- They run your applications, websites, APIs and other services.
- Like any server, they must be secured to prevent unauthorized access, data breaches and service disruption.



Amazon EC2

2. Secure EC2 Architecture (Example)



3. SSH Access and Secure Configuration

- Use key pair authentication instead of passwords.
- Restrict SSH access to trusted IP addresses only (using security groups).
- Use a non-root user and give sudo access when needed.
- Change the default SSH port if necessary.
- Disable password authentication.
- Use SSH config and keep your private key safe.

```
# Example: secure SSH connection
ssh -i my-key.pem ec2-user@your-ec2-ip
# Recommended SSH key perrissions
chmod 400 my-key.pem
```



4. Operating System Patching

- Regularly update the operating system to fix security vulnerabilities.
- Install security updates automatically where possible.
- Remove unused packages and services.
- Reboot when required to apply critical updates.

Example (Amazon Linux / Ubuntu)

Ubuntu

```
sudo apt update
sudo apt upgrade -y
```

Amazon Linux

```
sudo yum update -y
# or
sudo dnf update -y
```

5. Reducing Exposed Ports and Services

- Only open the ports that are required for your application.
- Use security groups (instance firewall) to control inbound and outbound traffic.
- Disable or uninstall unused services.
- Use a host firewall (e.g. ufw or firewalld) for additional control.
- Regularly review open ports and running services.

Security Group Rules (Example)

	Type	Protocol	Port	Source	Purpose
✓	SSH	TCP	22	203.0.113.0/24	Admin access (from trusted IP)
✓	HTTP	TCP	80	0.0.0.0/0	Web traffic (if needed)
✓	HTTPS	TCP	443	0.0.0.0/0	Secure web traffic
✗	All Other	All	All	0.0.0.0/0	Deny (not required)

6. Common Mistakes

- ✗ Leaving SSH open to the world (0.0.0.0/0).
- ✗ Using weak or shared SSH keys.
- ✗ Running instances without regular updates.
- ✗ Keeping unnecessary services and ports open.
- ✗ Using the default root user.
- ✗ Storing SSH private keys insecurely.
- ✗ Not monitoring instance logs and access.

7. Best Practices

- ✓ Allow SSH only from trusted IP addresses.
- ✓ Use key pair authentication and disable passwords.
- ✓ Keep the operating system and packages updated.
- ✓ Use the principle of least privilege.
- ✓ Close unused ports and services.
- ✓ Enable monitoring and logging (e.g. CloudWatch).
- ✓ Regularly review security group rules.

8. Key Takeaway



- Secure your EC2 instances from the internet.
- Use key-based SSH and restrict access.
- Keep the OS patched and remove unnecessary services.
- Only open the ports you need.
- Continuous monitoring helps detect and prevent threats.

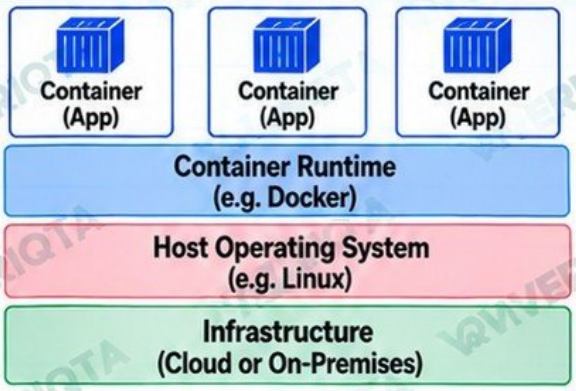
11. Container Security Basics

VERIQTA

1. What Is Container Security?

- Container security protects your container images, runtime, applications and data from threats.
- Containers share the host operating system, so poor configuration can affect the entire system.
- You need to secure the image, the container, and the runtime environment.
- Good container security reduces the risk of breaches and keeps your applications and data safe.

How Containers Fit in the Stack



2. Common Docker Security Risks

- Using untrusted or malicious images.
- Running containers as root.
- Exposing unnecessary ports.
- Storing secrets inside images.
- Using outdated images with known vulnerabilities.
- Mounting sensitive host directories.
- Running privileged containers.

3. Trusted Images and Vulnerability Scanning

- Always use official or trusted images from reliable sources (e.g. Docker Hub official images).
- Scan images for vulnerabilities before using them.
- Keep your base images updated.
- Use automated scanning tools in your CI/CD pipeline.

Example: Scan a Docker Image



4. Avoiding Privileged Containers

- Do not run containers in privileged mode (--privileged).
- Only give the specific capabilities your application needs.
- Avoid mounting the entire host filesystem (e.g. /).
- Use read-only filesystems where possible.
- Limit access to host resources (devices, kernel modules, etc.).

Privileged Container (Risky)

docker run --privileged ...



- Full access to host system.
- Can escape the container.
- High security risk.

Restricted Container (Safer)

docker run --cap-drop ALL ...



- Only specific permissions.
- Reduced attack surface.
- Safer for production.

5. Running Applications as Non-Root Users

- Do not run your application as the root user inside the container.
- Use a non-root user with the minimum required permissions.
- This limits the damage if the application is compromised.

Example: Add a Non-Root User in Dockerfile

```
1  FROM nginx:latest
2  RUN adduser -D appuser
3  USER appuser
4  CMD ["nginx","-g", "daemon off;"]
```



Runs as non-root user (appuser)

6. Best Practices

- Use trusted and up-to-date base images.
- Scan images for vulnerabilities.
- Avoid privileged containers.
- Run applications as non-root users.
- Limit exposed ports and network access.
- Do not store secrets in images.
- Use read-only filesystems where possible.
- Regularly update images and rebuild containers.

7. Key Takeaway



- Containers are powerful but can be risky if not secured.
- Use trusted images, scan for vulnerabilities, and keep them updated.
- Avoid privileged containers and run as non-root users.
- Follow security best practices to protect your applications.

12. Kubernetes Security Basics

VERIQTA

1. What Is Kubernetes Security?

- Kubernetes security protects your containers, workloads, cluster and data from unauthorized access and attacks.
- It helps you run applications safely in shared and multi-tenant environments.
- You need to secure the cluster, control access, restrict network traffic and protect workloads.
- Good security reduces the risk of compromised containers and data breaches.

Kubernetes (Security)

2. Securing Pods and Workloads

- Run containers as non-root users.
- Use minimal and trusted container images.
- Apply resource limits to prevent abuse.
- Use Pod security controls (e.g. Pod Security Standards).
- Avoid privileged containers and unnecessary capabilities.

Insecure Pod (Avoid)

```
securityContext:
  privileged: true
  runAsUser: 0
```

Runs as root with full privileges (high risk).

Secure Pod (Recommended)

```
securityContext:
  runAsNonRoot: true
  runAsUser: 1000
  allowPrivilegeEscalation: false
```

Runs as non-root with restricted privileges (safer).

3. Kubernetes RBAC (Role-Based Access Control)

- RBAC controls who can access Kubernetes resources and what they can do.
- Use roles and role bindings to grant only the required permissions.
- Avoid giving cluster-admin to everyone.
- Use service accounts for applications instead of user accounts.

How Kubernetes RBAC Works



4. NetworkPolicies and Pod Security Controls

- NetworkPolicies control the network traffic between pods.
- You can allow or reject traffic based on rules.
- Pod security controls (e.g. Pod Security Standards) help prevent risky configurations.
- These controls reduce the attack surface and limit lateral movement.

NetworkPolicy Example



5. Protecting Access to the Kubernetes API

- The Kubernetes API is the control plane for the cluster.
- Use strong authentication (certificates, tokens or OIDC).
- Restrict access to trusted IP addresses.
- Use RBAC to limit what users and service accounts can do.
- Enable audit logging to track API activity.

Kubernetes API Access (Example Flow)



6. Common Mistakes

- ✗ Giving cluster-admin to all users.
- ✗ Using default service accounts.
- ✗ Not using NetworkPolicies.
- ✗ Running privileged containers.
- ✗ Exposing the Kubernetes API to the internet.
- ✗ Not enabling audit logs.
- ✗ Using overly permissive roles and bindings.

7. Best Practices

- ✓ Follow the principle of least privilege.
- ✓ Use RBAC with specific roles.
- ✓ Enable NetworkPolicies.
- ✓ Run containers as non-root users.
- ✓ Use trusted and minimal images.
- ✓ Keep the cluster and add-ons updated.
- ✓ Monitor and audit access to the API.

8. Key Takeaway

- Secure your pods, network, access and the API.
- Use RBAC, NetworkPolicies and pod security controls.
- Run containers with least privilege.
- Protect the Kubernetes API and monitor activity.
- Layer multiple security controls for better protection.

13. Securing CI/CD Pipelines

VERIQTA

1. What Is CI/CD Pipeline Security?

- CI/CD pipeline security protects your code, build process, artifacts and deployments from attacks.
- Pipelines have access to sensitive resources like source code, credentials and production environments.
- If an attacker compromises your pipeline, they can deploy malicious code, steal secrets or take over your infrastructure.
- Good pipeline security reduces the risk of unauthorized changes and keeps your deployments safe.

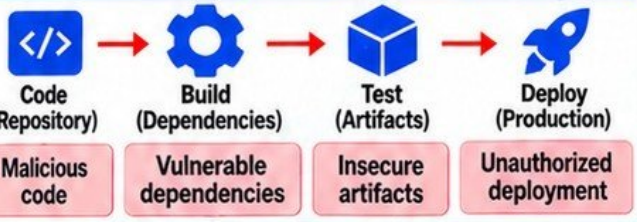


CI/CD Pipeline (Security)

2. Common Security Risks in CI/CD

- Exposed credentials (tokens, keys, passwords).
- Running untrusted code from pull requests.
- Using vulnerable dependencies and container images.
- Compromised build agents or runners.
- Overly permissive permissions to cloud resources.
- Malicious changes in pipeline configuration files.
- Lack of monitoring and auditing.

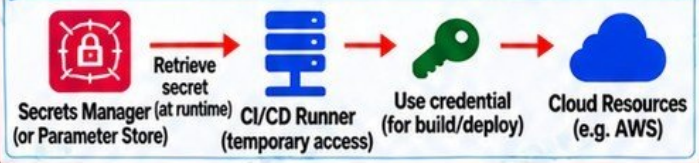
CI/CD Pipeline (Risk Points)



3. Protecting Pipeline Credentials

- Use a secrets manager (e.g. AWS Secrets Manager or Parameter Store).
- Do not store credentials in code or repository.
- Use short-lived credentials and rotate them regularly.
- Restrict who can access and use pipeline credentials.
- Use separate credentials for different environments (e.g. dev, staging, production).

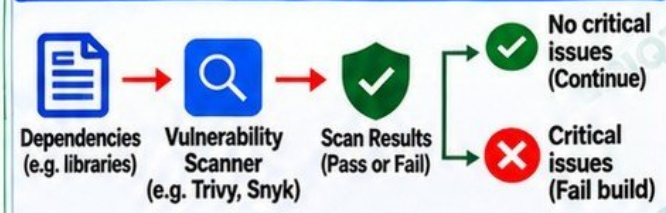
Secure Credential Flow



4. Dependency and Container Image Scanning

- Scan open source dependencies for known vulnerabilities.
- Scan container images before using them.
- Use trusted base images (e.g. official images).
- Fail the build if high or critical vulnerabilities are found.
- Keep dependencies and base images updated regularly.

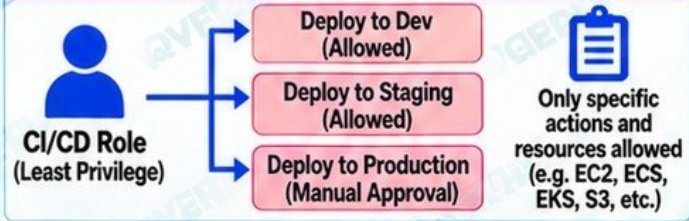
Scanning in the Pipeline



5. Restricting Deployment Permissions

- Use least privilege permissions for pipeline roles.
- Allow deployments only to specific environments.
- Use separate roles for build, test and deploy stages.
- Require manual approval for production deployments.
- Restrict who can modify pipeline configuration files.

IAM Permissions Example



6. Best Practices

- Use a secrets manager for all sensitive credentials.
- Scan dependencies and container images.
- Use trusted and minimal base images.
- Apply least privilege permissions to pipeline roles.
- Enable manual approvals for production.
- Monitor and audit pipeline activity.
- Keep your CI/CD tools and plugins updated.
- Restrict access to pipeline configuration files.

7. Key Takeaway

- Your CI/CD pipeline is a high-value target.
- Protect credentials, scan dependencies and images.
- Use least privilege permissions.
- Require approvals for production deployments.
- Monitor and audit your pipeline for suspicious activity.

14. Infrastructure as Code Security

VERIQTA

1. What Is Infrastructure as Code Security?

- Infrastructure as Code (IaC) security protects your Terraform configurations, state files and deployment process from security risks.
- It helps you detect and prevent insecure configurations before deployment.
- IaC defines real cloud resources, so mistakes can lead to serious security risks and high costs.
- Good IaC security ensures your infrastructure is secure, repeatable and compliant.



Terraform (IaC Security)

2. Common Security Risks in Terraform Configurations

- Hardcoded secrets (passwords, API keys, tokens).
- Overly permissive IAM roles and policies.
- Publicly accessible resources (e.g. S3 buckets, EC2).
- Exposing sensitive data in Terraform state files.
- Using outdated or vulnerable provider versions.
- Lack of input validation and security controls.
- Deploying resources without reviewing changes.

3. Avoiding Hardcoded Secrets

- Do not store secrets directly in Terraform files.
- Use environment variables, secret managers (AWS Secrets Manager, Parameter Store) or external tools.
- Keep sensitive values out of version control (e.g. Git).
- Use variables and reference secure values at runtime.

Bad Example (Hardcoded Secret)

```
resource "aws_db_instance" "example" {
  password = "MySecret123"
}
```

Good Example (Using Variable / Secret Manager)

```
resource "aws_db_instance" "example" {
  password = var.db_password
}
```

4. Detecting Insecure Infrastructure Definitions

- Use automated scanning tools to find security issues in your Terraform code.
- Check for public access, open ports and overly permissive permissions.
- Detect hardcoded secrets and vulnerable configurations.
- Validate configurations against security best practices and compliance standards.

Scanning Terraform Code (Example Tools)



Checkov (Open Source)



tfsec (Open Source)



Terrascan (Open Source)



Snyk (Commercial)

5. Reviewing Infrastructure Changes Before Deployment

- Always review Terraform plan output.
- Use pull requests and code reviews for all changes.
- Check for unintended or risky changes (e.g. public access).
- Require approvals before deployment to production.

Check Changes with Terraform Plan

```
1  terraform init
2  terraform plan
3  # Review the changes before applying
4  terraform apply  # only after approval
```



6. Example: Detecting Public Resources

Risky Configuration

```
resource "aws_s3_bucket" "example" {
  bucket = "my-public-bucket"
  acl    = "public-read"
}
```

Secure Configuration

```
resource "aws_s3_bucket" "example" {
  bucket = "my-private-bucket"
  acl    = "private"
  block_public_access = true
}
```

7. Common Mistakes

- Storing secrets in Terraform files.
- Making resources public by default.
- Using overly permissive IAM roles.
- Not reviewing plan output.
- Ignoring security scanning tools.
- Deploying without approvals.
- Using outdated provider versions.

8. Best Practices

- Use variables and secret managers.
- Scan Terraform code for vulnerabilities.
- Follow least privilege principles.
- Review plan output before applying.
- Use pull requests and code reviews.
- Enable automated checks in CI/CD.
- Keep providers and modules updated.

9. Key Takeaway



- Your Terraform code can create both secure and insecure infrastructure.
- Avoid hardcoded secrets and public resources.
- Use scanning tools and review changes before deployment.
- Follow security best practices to keep your infrastructure safe.

15. Cloud Application and API Security

VERIQTA

1. What Is Cloud Application and API Security?

- Cloud application and API security protects your applications, APIs and data from unauthorized access and attacks.
- It ensures secure communication, proper authentication and authorization.
- It helps you validate user input, prevent abuse and keep your services available and reliable.
- Good application and API security reduces the risk of data breaches, service disruption and financial loss.



Cloud Application and API Security

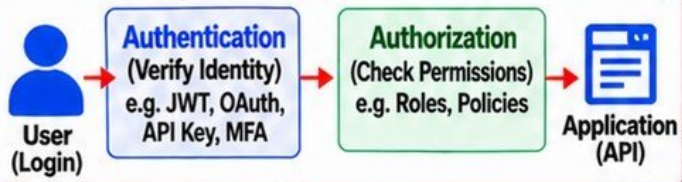
2. HTTPS and Secure Communication

- Use HTTPS (TLS) to encrypt data between clients and your application.
- TLS prevents attackers from reading or modifying data in transit.
- Always use valid and trusted SSL/TLS certificates (e.g. from a trusted CA).
- Redirect all HTTP traffic to HTTPS.
- Use strong TLS versions (TLS 1.2 or TLS 1.3) and disable weak ciphers.



3. Authentication and Authorization

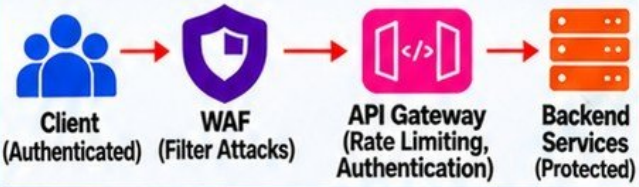
- Authentication verifies who the user is (e.g. username/password, API key, JWT, OAuth).
- Authorization controls what the user can do (e.g. roles and permissions).
- Use strong authentication methods (e.g. MFA for admin access).
- Follow the principle of least privilege (give only the permissions needed).



4. Protecting APIs from Unauthorized Access

- Use API keys, OAuth 2.0 or JWT for authenticated access.
- Restrict access to trusted users and services only.
- Use API Gateway or WAF to filter and block malicious requests.
- Implement rate limiting and throttling to prevent abuse.
- Monitor and log API activity for suspicious behavior.

API Security Layers (Example)



5. Input Validation and Basic Rate Limiting

- Validate all user input on the server side.
- Reject invalid or unexpected input to prevent attacks (e.g. SQL injection, command injection).
- Use rate limiting to prevent too many requests from the same user or IP address.
- Return clear error messages without exposing sensitive information.

Input Validation (Example)

```
# Check if input is valid
if not username.isalnum():
    return "Invalid input"
```

Rate Limiting (Example)

```
# Limit requests per minute
RateLimit: 100 requests
per IP per minute
(excess requests are blocked)
```

6. Common Mistakes

- Using HTTP instead of HTTPS.
- Exposing APIs without authentication.
- Using weak authentication methods.
- Giving excessive permissions.
- Not validating user input.
- Not implementing rate limiting.
- Returning sensitive information in error messages.
- Not monitoring or logging API activity.

7. Best Practices

- Always use HTTPS with strong TLS.
- Implement proper authentication and authorization.
- Use API Gateway, WAF and rate limiting.
- Validate and sanitize all user input.
- Follow the principle of least privilege.
- Monitor and log API activity.
- Return generic error messages.
- Regularly review and test your application and API security.

8. Key Takeaway

- Secure communication, strong authentication, proper authorization, input validation and rate limiting keep your applications and APIs safe.
- Protect your APIs from unauthorized access and abuse.
- Follow security best practices to build reliable and secure services.

16. Cloud Logging and Monitoring

VERIQTA

1. Why Security Logs Matter?

- Security logs record what happened in your cloud environment.
- They help you detect suspicious activity, investigate incidents and find the root cause.
- Logs provide evidence for security investigations and compliance.
- Without logs, attackers can make changes without being detected.



Security Logs (Protection & Visibility)

2. AWS CloudTrail and CloudWatch

- AWS CloudTrail records API calls and account activity.
- CloudTrail helps you see who did what, when and from where.
- Amazon CloudWatch collects logs, metrics and alarms from your AWS resources.
- Use both services together to monitor your infrastructure and detect security issues.

AWS CloudTrail

- Records API calls
- Tracks user activity
- Shows who, what, when, where
- Helps with incident investigation

Amazon CloudWatch

- Collects logs and metrics
- Monitors resource health
- Creates alarms
- Helps you detect unusual activity

3. Identifying Suspicious Activity

- Look for unusual login attempts or failed authentication.
- Check for API calls from unknown locations or unusual IP addresses.
- Monitor for unexpected changes to your resources (e.g. new users, security groups).
- Look for unusual spikes in traffic, errors or resource usage.

Examples of Suspicious Activity



Multiple failed logins



Login from unknown location



Unexpected resource changes

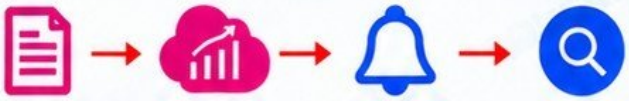


Unusual traffic or API calls

4. Understanding Basic Security Alerts

- CloudWatch can create alarms when a metric or log pattern looks unusual.
- Alerts help you respond quickly to potential threats.
- Common alerts include multiple failed logins, unusual API activity and high error rates.
- Investigate alerts to confirm if the activity is legitimate or malicious.

CloudWatch Alert Flow (Example)



Logs & Metrics (e.g. API calls, errors, CPU) → CloudWatch Alarm (Detects Issue) → Notification (e.g. Email, SNS) → Investigate (Confirm and Take Action)

5. Example: Viewing CloudTrail Logs

1. Open the AWS CloudTrail console.
2. Select Event history.
3. Look for important events (e.g. ConsoleLogin, CreateUser, DeleteSecurityGroup).
4. Check the time, user, source IP and event details.

Sample CloudTrail Event (Simplified)

```
{
  "eventName": "ConsoleLogin",
  "eventTime": "2026-10-14T10:24:36Z",
  "userIdentity":{ "userName": "ann.admin" },
  "sourceIPAddress":"203.0.113.10",
  "eventSource": "signin.amazonaws.com"
}
```

6. Common Mistakes

- Not enabling CloudTrail for all regions.
- Not monitoring logs regularly.
- Ignoring or delaying security alerts.
- Not setting up CloudWatch alarms.
- Keeping logs for too short a period.
- Not investigating unusual activity.
- Assuming all activity is legitimate.

7. Best Practices

- Enable CloudTrail in all regions.
- Send logs to a secure S3 bucket.
- Set up CloudWatch alarms for important events.
- Monitor logs and alerts regularly.
- Keep logs for the required retention period.
- Investigate and respond to suspicious activity quickly.
- Use least privilege access and monitor user activity.

8. Key Takeaway



- Security logs give you visibility into what happens in your cloud environment.
- Use CloudTrail and CloudWatch to monitor and detect suspicious activity.
- Set up alerts and investigate unusual events.
- Logs help you respond quickly and keep your cloud infrastructure secure.

17. Cloud Threat Detection

VERIQTA 17

1. What Is Cloud Threat Detection?

- Cloud threat detection identifies suspicious or malicious activity in your cloud environment.
- It continuously analyzes logs, network traffic, user behavior and API activity.
- It helps you detect potential security threats early before they cause damage.
- Good threat detection improves visibility, response time and overall cloud security.



Cloud Threat Detection

2. AWS GuardDuty and Security Hub

- AWS GuardDuty uses machine learning to detect threats like unusual API activity, malicious IPs and suspicious behavior.
- AWS Security Hub gives you a central view of security findings from AWS services and partner tools.
- Both services help you identify and respond to security threats in your AWS environment.
- They work together to improve threat detection, investigation and compliance.



AWS GuardDuty

- Detects malicious activity
- Uses machine learning
- Monitors logs, API calls and network activity



AWS Security Hub

- Central view of security findings
- Aggregates findings from AWS and partner tools
- Helps with compliance and prioritization

3. Unusual Logins and Suspicious API Activity

- Unusual logins include access from unknown locations, new devices or impossible travel.
- Suspicious API activity includes unexpected API calls, changes to critical resources or unusual data access.
- These activities can be siigns of compromised accounts or malicious behavior.
- Monitoring and alerting on these activities helps you detect threats early.

Examples of Suspicious Activity



Login from unknown location



New device or unusual login time



Unusual API calls



Unexpected changes to resources

4. Understanding Findings Before Taking Action

- Read the finding details carefully (what happened, when, where, and which resource).
- Check the severity level (e.g. low, medium, high, critical).
- Look at supporting information (IP addresses, user, API calls, affected resources).
- Determine if it is a real threat or a false positive.
- Investigate and confirm before taking action to avoid unnecessary disruption.

Anatomy of a Security Finding (Example)



Finding type: UnauthorizedAccess:IAMUser
Severity: High
Description: Unusual API activity from a known malicious IP address.
Resource: IAM User (dev-user)
Time: 2026-10-14 03:24:19 UTC
Source IP: 203.0.113.50 (unknown location)
Action: Review activity and take appropriate action.

5. Threat Detection Flow (Simplified)



6. Common Mistakes

- Ignoring security findings.
- Not investigating unusual logins.
- Assuming all findings are false positives.
- Taking action without checking the details.
- Not enabling GuardDuty or Security Hub.
- Not monitoring findings regularly.

7. Best Practices

- Enable GuardDuty in all regions.
- Use Security Hub for central visibility.
- Monitor unusual logins and API activity.
- Review findings regularly.
- Investigate before taking action.
- Set up alerts for high and critical findings.
- Keep your environment updated and secured.

8. Key Takeaway



- Cloud threat detection helps you identify and respond to suspicious or malicious activity early.
- AWS GuardDuty detects threats using machine learning, and Security Hub gives you a central view of findings.
- Monitor unusual logins and API activity.
- Always review findings carefully before taking action.
- Use alerts and best practices to keep your cloud environment secure.

18. Cloud Security Incident Response

VERIQTA

1. What Is Cloud Security Incident Response?

- Cloud security incident response is the process of detecting, responding to and recovering from security incidents in your cloud environment.
- It helps you minimize damage, reduce downtime and prevent future incidents.
- A good incident response process ensures a structured and calm approach when something goes wrong.
- It protects your data, systems, customers and business reputation.



Cloud Security Incident Response

2. Identifying and Assessing an Incident

- Look for unusual activity (e.g. unexpected logins, API calls, resource changes).
- Use monitoring tools (CloudWatch, GuardDuty, Security Hub) to detect alerts.
- Confirm if it is a real security incident or a false alarm.
- Assess the impact (which resources, what data, how serious).
- Determine the scope (single resource or multiple resources).

Incident Detection Flow



3. Containing Compromised Resources

- Take immediate action to limit further damage.
- Isolate or stop compromised resources (e.g. EC2 instance, container, IAM user).
- Revoke or rotate compromised credentials (e.g. access keys, tokens).
- Block malicious IP addresses using security groups or WAF.
- Apply temporary restrictions while investigating.

Containment Examples



4. Preserving Logs and Evidence

- Save and protect logs from deletion or modification.
- Collect logs from CloudTrail, CloudWatch, VPC Flow Logs and application logs.
- Keep a record of important events, timestamps and affected resources.
- Store evidence in a secure location (e.g. S3 bucket with restricted access).
- Preserving evidence helps with investigation, compliance and future prevention.

Log and Evidence Sources



5. Recovering Safely and Verifying Changes

- Remove the root cause (e.g. malware, unauthorized access).
- Restore affected resources from clean backups if needed.
- Apply security fixes and update configurations.
- Verify that the environment is secure before returning to normal operations.
- Monitor closely after recovery to detect any unusual activity.

Recovery and Verification Flow



6. Common Mistakes

- ✗ Ignoring early warning signs.
- ✗ Taking action without understanding the impact.
- ✗ Not preserving logs and evidence.
- ✗ Deleting logs by mistake.
- ✗ Not rotating compromised credentials.
- ✗ Restoring from an infected backup.
- ✗ Returning to normal operations too quickly.
- ✗ Not monitoring after recovery.

7. Best Practices

- ✓ Have an incident response plan and run regular drills.
- ✓ Use monitoring tools (CloudWatch, GuardDuty, Security Hub).
- ✓ Follow the principle of least privilege.
- ✓ Preserve logs and evidence before making changes.
- ✓ Communicate and document all actions.
- ✓ Verify security before full recovery.
- ✓ Continuously monitor after the incident.

8. Key Takeaway

- Detect incidents early, contain the damage, preserve evidence and recover safely.
- Use AWS security services and follow a structured response process.
- Always verify and monitor after recovery to prevent recurrence.
- A good incident response process keeps your cloud environment secure and resilient.

19. Common Cloud Security Mistakes

VERIQTA

1. Common Cloud Security Mistakes

- Simple misconfigurations can expose your cloud resources, data and applications.
- Attackers often take advantage of common mistakes to gain unauthorized access.
- Understanding these mistakes helps you prevent them and keep your cloud environment secure.
- Always review your configurations and follow security best practices.



Cloud Security Mistakes Can Be Costly

2. Publicly Exposed Storage

- Leaving storage (e.g. S3 buckets) open to the public can expose sensitive data.
- Anyone on the internet can read or download the files.
- This can lead to data breaches, financial loss and reputational damage.

Example: Public S3 Bucket (Risky)

Bucket policy allows public read access.

```
{
  "Effect": "Allow",
  "Principal": "*",
  "Action": "s3:GetObject",
  "Resource": "arn:aws:s3:::example-bucket/*"
}
```

3. Overly Permissive IAM Policies

- Giving users, roles or services more permissions than needed increases the risk of misuse.
- Attackers can take advantage of excessive permissions to access and modify resources.
- Always follow the principle of least privilege.

Overly Permissive Policy (Risky)

Full access to all AWS services.

```
{
  "Effect": "Allow",
  "Action": "*",
  "Resource": "*"
}
```

Least Privilege Policy (Better)

Only required permissions.

```
{
  "Effect": "Allow",
  "Action": ["s3:GetObject"],
  "Resource": "arn:aws:s3:::
    example-bucket/*"
}
```

4. Hardcoded Credentials

- Storing API keys, access keys or passwords in code, configuration files or containers is risky.
- If the code is shared or exposed, attackers can steal the credentials.
- Use environment variables, secret managers (e.g. AWS Secrets Manager) or IAM roles instead.

Hardcoded Credentials (Risky)

```
aws_access_key = "AKIA..."
aws_secret_key = "abcd123...",
password = "MyPassword123"
```

Use Secure Methods (Better)

- Environment variables
  AWS Secrets Manager
- IAM roles (for EC2/ECS/Lambda)
- Parameter Store

5. Open Management Ports

- Leaving management ports (e.g. SSH 22, RDP 3389) open to the internet allows unauthorized access.
- Attackers can brute force or exploit vulnerabilities to gain control.
- Restrict access to trusted IP addresses and use a VPN or bastion host.



6. Missing Logs and Unpatched Workloads

- Not enabling logs makes it hard to detect and investigate security incidents.
- Unpatched operating systems and applications can have known vulnerabilities.
- Attackers often exploit unpatched workloads to gain access.

Enable Logging (Better)

CloudTrail (API activity)

CloudWatch (Logs and metrics)

Keep Workloads Updated

OS Updates (Security patches)

Application Updates (Fix vulnerabilities)

7. How to Investigate These Mistakes

1. Check your cloud configuration (e.g. S3, IAM, security groups).
2. Review access logs (CloudTrail, CloudWatch, application logs).
3. Look for unusual activity (unexpected logins, API calls, data access).
4. Identify the root cause (misconfiguration, exposed credentials, etc.).
5. Assess the impact (what resources and data were affected).
6. Fix the issue and apply security best practices.

8. How to Correct These Mistakes

- Make storage private and restrict access.
- Use least privilege IAM policies.
- Remove hardcoded credentials and use secure methods (e.g. Secrets Manager, IAM roles).
- Restrict management ports to trusted IP addresses.
- Enable logging and monitoring.
- Keep operating systems and applications updated.
- Regularly review your cloud environment for misconfigurations.

9. Key Takeaway

- Most cloud security incidents start with simple mistakes.
- Regularly review your configurations, logs and access permissions.
- Investigate and fix issues before attackers can exploit them.
- Follow security best practices to keep your cloud environment secure and resilient.

20. Your First Cloud Security Practice Lab

1. Lab Overview

- In this lab, you will set up a small and isolated AWS environment to practice cloud security basics.
- You will create a restricted IAM role, enable security logging, create a private S3 bucket with public access blocked, verify access controls and inspect logs.
- Finally, you will clean up all resources to avoid unexpected charges.



Your First Cloud Security Practice Lab

2. Create an Isolated AWS Practice Environment

1. Use a dedicated AWS account (preferably a free tier account) or an isolated environment.
2. Keep only the resources for this lab.
3. Use a consistent naming convention (e.g. veriqta-lab-<yourname>).
4. Enable a default region (e.g. us-east-1).



3. Configure a Restricted IAM Role

1. Create an IAM role with limited permissions (e.g. allow only S3 and CloudWatch).
2. Avoid using the root user for this lab.
3. Attach a managed or custom policy with least privilege access.
4. Test the role to confirm it can only perform the allowed actions.

Example IAM Policy (Allow S3 Read Only)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["s3:ListBucket", "s3:GetObject"],
      "Resource": "arn:aws:s3:::veriqta-lab-bucket/*"
    }
  ]
}
```

Avoid Overly Permissive Policies

Do NOT use:

- AdministratorAccess
- "Action": "*"
- "Resource": "*".

4. Enable Basic Security Logging

1. Enable AWS CloudTrail to log API activity.
2. Enable Amazon CloudWatch to collect and view logs.
3. Keep the default settings for this lab.
4. Verify that logs are being delivered correctly.



AWS CloudTrail

- Records API calls
- Tracks user activity
- Helps with investigation



Amazon CloudWatch

- Collects logs and metrics
- Monitor resource activity
- Create simple log groups
- View logs in real time

5. Create a Private S3 Bucket with Public Access Blocked

1. Create an S3 bucket with a unique name (e.g. veriqta-lab-bucket-<yourname>).
2. Enable Block Public Access (all settings).
3. Do not allow public read or write access.
4. Upload a test file and verify it is private.

S3 Block Public Access Settings (Recommended)



Amazon S3 (Private Bucket)

- Block public access to buckets and objects
- Block public access to new access control lists (ACLs)
- Block public access to any access control lists (ACLs)
- Block public access to new public bucket or access point policies
- Block public and cross-account access through any public bucket or access point policies

6. Verify Access Controls and Inspect Logs

1. Try to access the S3 bucket without permission (should be denied).
2. Access the bucket using your restricted IAM role (should be allowed).
3. Check CloudTrail for the API calls (e.g. S3 access attempts).
4. View the logs in CloudWatch (if enabled).

Example: Check S3 Access in CloudTrail



Event name: `GetObject`
Event source: `s3.amazonaws.com`
Error code (if denied): `AccessDenied`
Look for the `user`, `role`, source IP and timestamp.

7. Clean Up Resources and Confirm No Unexpected Charges

1. Delete the S3 bucket and all objects.
2. Disable or remove the IAM role and policies (if not needed).
3. Check that CloudTrail and CloudWatch costs are minimal (free tier).
4. Verify in the AWS Billing console that there are no unexpected charges.

8. Lab Flow Summary



9. Key Takeaway



- A secure cloud environment starts with least privilege access, proper logging and correct configuration.
- Always verify access controls and review logs to understand what is happening.
- Clean up resources after labs to avoid unexpected charges.
- These practices help you build real world cloud security habits.